

Build a Complete Full-Stack Person Management System

You are an expert full-stack software engineer and UI/UX developer.

Build a **production-ready, responsive, secure multi-user Person Management System** based on the requirements below.

The application should be designed cleanly enough to be delivered to a real freelance client.

1. PROJECT OBJECTIVE

Build a web application where authorized users can:

1. Log in securely.
2. Add person details.
3. Upload a person's photo.
4. Select State and dynamically load Districts belonging to that State.
5. View persons state-wise.
6. View persons district-wise.
7. Search persons.
8. Paginate large datasets.
9. Edit/delete persons according to their role.
10. Export filtered person data to CSV/PDF.
11. Manage users according to their permissions.

The application must have a professional business-style UI and should be easy for a non-technical client to use.

2. TECHNOLOGY STACK

Use the following stack unless there is a strong technical reason to change it:

Frontend

- Next.js
- React
- TypeScript
- Tailwind CSS
- Responsive design
- Modern reusable components

Backend

Use Django REST Framework.

- Python
- Django
- Django REST Framework
- JWT authentication
- Password hashing
- REST APIs

Database

Use PostgreSQL.

Design the database using proper relationships, foreign keys, indexes, constraints, and timestamps.

File Storage

Person photos should be stored using a proper file-storage mechanism.

For development:

- Local media storage.

For production:

- Structure the application so cloud storage such as AWS S3 can be added easily.

3. PROJECT ARCHITECTURE

Use a clean separation:

Frontend:

Next.js → API calls → Django REST API → PostgreSQL

Do NOT put database logic directly inside the frontend.

All database operations must go through backend APIs.

Use environment variables for:

- Database credentials
- JWT configuration
- API URLs
- Storage configuration

- Other secrets

Never hard-code passwords, secrets, JWT keys, or database credentials.

4. DATABASE DESIGN

Create the following main entities.

Users

Fields:

- id
- username
- full_name
- email
- phone
- password_hash
- role
- is_active
- created_at
- updated_at

Roles:

- ADMIN
- USER

Admin permissions:

- Add persons
- View persons
- Edit persons
- Delete persons
- Search persons
- Filter persons
- Export data
- Manage users

User permissions:

- View persons
- Search persons
- Filter persons
- Export data if permitted

Users must NOT be able to access admin-only APIs.

5. STATES TABLE

Create:

States

Fields:

- id
- state_name
- state_code
- is_active
- created_at
- updated_at

Requirements:

- state_name must be unique.
 - state_code should be unique.
 - Only active states should normally appear in dropdowns.
-

6. DISTRICTS TABLE

Create:

Districts

Fields:

- id
- state_id
- district_name
- district_code
- is_active
- created_at
- updated_at

Relationship:

One State → Many Districts

Requirements:

- district_name should be unique within a state.
 - District API must support filtering by state_id.
 - Only active districts should appear in normal dropdowns.
-

7. PERSONS TABLE

Create:

Persons

Fields:

- id
- name
- phone
- email
- address
- state_id
- district_id
- photo
- created_by
- updated_by
- created_at
- updated_at
- is_active

Relationships:

State:

State 1 → Many Persons

District:

District 1 → Many Persons

User:

User 1 → Many Persons through created_by.

8. IMPORTANT DATABASE RULES

Use foreign keys.

Do not store state name or district name repeatedly inside the Persons table.

Store:

state_id

district_id

Use joins/relationships when retrieving the names.

Add indexes for:

- state_id
- district_id
- phone
- email
- created_at

Add appropriate unique constraints where required.

Prevent a person from being assigned a district belonging to another state.

Example:

If State = Andhra Pradesh

District must belong to Andhra Pradesh.

The backend must validate this even if the frontend already validates it.

9. AUTHENTICATION

Implement secure JWT authentication.

Required APIs:

POST /api/auth/login

POST /api/auth/refresh

POST /api/auth/logout

GET /api/auth/me

Login should accept:

- username/email
- password

On successful login:

- Return access token.
- Return refresh token using a secure approach.
- Return user information and role.

Do not store plain-text passwords.

Use Django's secure password hashing mechanism.

10. AUTHORIZATION

Implement role-based authorization.

Backend must verify the user's role for every protected operation.

Example:

ADMIN:

POST /api/persons

PUT /api/persons/{id}

DELETE /api/persons/{id}

USER:

GET /api/persons

GET /api/persons/{id}

Users must receive HTTP 403 when attempting admin-only operations.

Do not rely only on frontend hiding buttons.

Authorization must be enforced on the backend.

11. LOGIN PAGE

Create a professional login screen.

Fields:

- Email/Username
- Password

Features:

- Show/hide password
- Validation
- Loading state
- Error messages
- Invalid credentials message

- Remember appropriate authentication state securely
- Redirect after successful login

After login:

ADMIN → Admin Dashboard

USER → Person Listing Dashboard

12. DASHBOARD

Create a clean dashboard.

Display summary cards such as:

- Total Persons
- Total States
- Total Districts
- Active Users

For admin:

Also display:

- Recently Added Persons
- Quick Add Person button
- User Management

Keep the dashboard simple and professional.

Do not overload the screen with unnecessary charts.

13. PERSON FORM

Create an Add/Edit Person form.

Fields:

Personal Information

Name

Phone Number

Email

Address

Photo

Location

State

District

14. DYNAMIC STATE → DISTRICT DROPDOWN

This is an important requirement.

When the user selects a State:

1. Clear the existing District selection.
2. Call the district API.
3. Load districts belonging only to the selected state.
4. Display them in the District dropdown.

Example:

State:

Andhra Pradesh

District:

Nellore

Naidupeta etc. should only appear if they exist in the selected state's district list.

When State changes:

- District must reset.
- Old district must not remain selected.
- New district list must load.

If no state is selected:

District dropdown should be disabled.

Show:

"Select State first"

while disabled.

15. FORM VALIDATION

Frontend validation:

Name:

- Required
- Minimum reasonable length
- Maximum reasonable length

Phone:

- Required
- Validate Indian phone number format.
- Exactly 10 digits where applicable.

Email:

- Required
- Valid email format

Address:

- Required

State:

- Required

District:

- Required

Photo:

- Optional or required according to the final configuration.

Photo validation:

- Allow JPG/JPEG/PNG/WebP.
- Validate file size.
- Reject unsupported file types.

Backend must repeat all important validation.

Never trust frontend validation alone.

16. PHOTO UPLOAD

Implement photo upload.

Requirements:

- Preview image before submission.
- Allow user to remove/change image.
- Validate file type.
- Validate file size.
- Generate a safe filename.
- Do not trust the original filename.
- Store photo path/URL in database.
- Display thumbnail in person listing.

If editing a person:

- Keep existing photo if no new photo is uploaded.
- Replace the old photo when a new photo is uploaded.
- Handle old file cleanup appropriately.

17. PERSON LISTING PAGE

Create a professional table/grid.

Columns:

- Photo
- Name
- Phone
- Email
- Address
- State
- District
- Created Date
- Actions

Actions:

View

Edit

Delete

Edit/Delete should only appear for ADMIN users.

USER should have View only.

18. SEARCH

Implement server-side search.

Search should support:

- Name
- Phone
- Email
- Address

Example:

User types:

Chandu

API should return matching persons.

Use debouncing on the frontend so unnecessary API requests are avoided.

Show:

"No persons found"

when there are no results.

19. STATE-WISE FILTER

Add a State filter.

Example:

All States

Andhra Pradesh

Telangana

Karnataka

Tamil Nadu

etc.

When a state is selected:

Display only persons belonging to that state.

API example:

GET /api/persons?state_id=1

20. DISTRICT-WISE FILTER

Add dependent District filter.

Flow:

Select State

↓

Load Districts

↓

Select District

↓

Load persons belonging to that district.

API example:

GET /api/persons?state_id=1&district_id=10

District dropdown must remain disabled until a State is selected.

21. COMBINED FILTERS

Support combinations.

Examples:

Search:

"Ravi"

State:

Andhra Pradesh

District:

Nellore

The backend should apply all filters together.

Possible query:

GET /api/persons?search=Ravi&state_id=1&district_id=10

Do not fetch the entire database and filter only in React.

Filtering should happen on the server.

22. PAGINATION

Implement server-side pagination.

Default:

20 records per page.

Allow:

10

20

50

100

Display:

- Current page
- Total records
- Total pages
- Previous
- Next

Example:

Showing 21–40 of 245 persons

Do not load thousands of records into the browser at once.

23. SORTING

Allow sorting by useful columns:

- Name
- Created Date
- State
- District

Use server-side sorting.

Example:

GET /api/persons?ordering=name

Prevent arbitrary unsafe SQL ordering parameters.

24. VIEW PERSON

Create a detailed person profile page/modal.

Display:

Large photo

Name

Phone

Email

Address

State

District

Created By

Created Date

Updated Date

For admin:

Edit

Delete

buttons.

25. EDIT PERSON

Admin can edit existing person details.

Requirements:

- Load existing data.
 - Automatically select current State.
 - Load districts belonging to that state.
 - Automatically select current District.
 - Allow changing state.
 - If state changes, reset district.
 - Allow changing photo.
 - Validate all fields.
-

26. DELETE PERSON

Only ADMIN can delete.

Before deleting:

Show confirmation:

"Are you sure you want to delete this person?"

Do not immediately delete after button click.

Prefer soft deletion using:

`is_active = false`

instead of permanently deleting data.

Normal listing should show only active persons.

27. EXPORT CSV

Implement CSV export.

Export should respect current filters.

Example:

If the user selects:

State = Andhra Pradesh

District = Nellore

Search = Ravi

CSV should contain only the filtered results.

Columns:

Name

Phone

Email

Address

State

District

Created Date

Do not export unnecessary internal fields.

28. EXPORT PDF

Implement PDF export.

PDF should include:

Application title

Applied filters

Generated date/time

Person records

Columns:

Name

Phone

Email

Address

State

District

Use a clean professional layout.

Handle large datasets properly.

If necessary, generate the PDF on the backend rather than the browser.

29. USER MANAGEMENT

Admin-only page.

Display:

- Full Name
- Username
- Email
- Phone
- Role
- Status
- Created Date

Admin should be able to:

- Create user
- Edit user
- Activate/deactivate user
- Change role
- Reset password if required

Do not allow users to access this page.

30. API STRUCTURE

Create clean REST APIs.

Suggested structure:

Authentication:

POST /api/auth/login

POST /api/auth/refresh

POST /api/auth/logout

GET /api/auth/me

States:

GET /api/states

GET /api/states/{id}

Districts:

GET /api/districts?state_id={id}

Persons:

GET /api/persons

GET /api/persons/{id}

POST /api/persons

PUT /api/persons/{id}

DELETE /api/persons/{id}

CSV:

GET /api/persons/export/csv

PDF:

GET /api/persons/export/pdf

Users:

GET /api/users

POST /api/users

PUT /api/users/{id}

DELETE /api/users/{id}

Dashboard:

GET /api/dashboard/summary

31. API RESPONSE FORMAT

Use a consistent response structure.

Success:

```
{ "success": true, "message": "Persons retrieved successfully", "data": [], "pagination": {} }
```

Error:

```
{ "success": false, "message": "Validation failed", "errors": {} }
```

Use appropriate HTTP status codes:

200

201

400

401

403

404

409

500

32. SECURITY REQUIREMENTS

Implement:

- JWT authentication
- Password hashing
- Role-based authorization
- Backend validation
- CORS configuration

- CSRF protection where applicable
- Secure file upload validation
- SQL injection protection through ORM/query parameterization
- XSS-safe rendering
- Rate limiting for login if practical
- Environment variables for secrets
- No passwords in logs
- No sensitive information in frontend source code

Do not expose database credentials.

Do not expose secret keys.

Do not trust user-provided file extensions.

33. UI/UX REQUIREMENTS

Design should look like a professional business application.

Use:

- Sidebar navigation
- Top navigation/header
- Dashboard cards
- Clean tables
- Modern forms
- Consistent buttons
- Toast notifications
- Confirmation dialogs
- Loading skeletons
- Empty states
- Error states

Responsive design:

Desktop

Laptop

Tablet

Mobile

The application should remain usable on smaller screens.

34. SIDEBAR MENU

For ADMIN:

Dashboard

Persons

Add Person

State-wise View

District-wise View

Users

Profile

Logout

For USER:

Dashboard

Persons

State-wise View

District-wise View

Profile

Logout

Hide admin-only navigation items from normal users.

Again, hiding the UI is NOT the security mechanism. Backend authorization is mandatory.

35. PERSON PAGE DESIGN

At the top:

Person Management

[Add Person]

Below:

Search box

State dropdown

District dropdown

[Reset Filters]

[Export CSV]

[Export PDF]

Then:

Person table

Pagination

Use sticky table headers if useful.

36. FILTER RESET

Create a Reset button.

When clicked:

- Clear search.
 - State = All.
 - District = All.
 - Reset pagination to page 1.
 - Reload data.
-

37. LOADING STATES

Every API operation should have a loading state.

Examples:

Login:

"Signing in..."

Person save:

"Saving..."

Person delete:

"Deleting..."

District loading:

"Loading districts..."

Person table:

Skeleton/loading indicator.

Do not leave the interface looking frozen.

38. ERROR HANDLING

Handle:

Network errors

401 Unauthorized

403 Forbidden

404 Not Found

400 Validation errors

500 Server errors

Display user-friendly messages.

Do not display raw stack traces to users.

39. EMPTY STATES

If no persons exist:

"No persons have been added yet."

Provide:

"Add Person"

button for admins.

If search/filter returns nothing:

"No persons found matching your filters."

Provide:

"Clear Filters"

button.

40. AUDIT INFORMATION

For person records maintain:

created_by

updated_by

created_at

updated_at

This will allow the client to know who created or modified records.

41. SEED DATA

Create initial seed data.

Include:

- Sample admin account
- Sample normal user
- Indian states
- Districts for the states

Do not hard-code state/district data into React.

State and district data must come from the database/API.

Clearly document the sample login credentials in the development README only.

Do not use weak credentials in production.

42. DATABASE MIGRATIONS

Create proper Django migrations.

The project must be reproducible from a fresh database.

Provide commands/documentation for:

- Installing dependencies
 - Creating database
 - Running migrations
 - Creating seed data
 - Running backend
 - Running frontend
-

43. PROJECT STRUCTURE

Use a clean structure.

Example:

frontend/

app/

components/

services/

hooks/

types/

utils/

public/

backend/

manage.py

config/

apps/

```
authentication/
```

```
users/
```

```
persons/
```

```
locations/
```

```
dashboard/
```

```
media/
```

```
requirements.txt
```

```
README.md
```

Adjust the structure if needed, but maintain clear separation of responsibilities.

44. FRONTEND API SERVICE

Create a centralized API service.

Do not scatter fetch/API logic throughout components.

Create reusable functions such as:

```
login()
```

```
getCurrentUser()
```

```
getStates()
```

```
getDistricts()
```

```
getPersons()
```

```
getPerson()
```

```
createPerson()
```

```
updatePerson()
```

```
deletePerson()
```

```
exportCSV()
```

exportPDF()

getDashboardSummary()

Use TypeScript types for API responses.

45. REUSABLE COMPONENTS

Create reusable components where appropriate:

Button

Input

Select

Modal

ConfirmDialog

DataTable

Pagination

SearchInput

FileUpload

LoadingSpinner

EmptyState

Toast

ProtectedRoute/AuthGuard

Do not duplicate the same UI logic unnecessarily.

46. TYPESCRIPT

Use TypeScript properly.

Create interfaces/types for:

User

Person

State

District

Pagination

APIResponse

DashboardSummary

Do not use "any" everywhere.

Avoid unnecessary type casting.

47. PERFORMANCE

The application should work efficiently with large datasets.

Important:

- Server-side pagination
 - Server-side filtering
 - Server-side search
 - Database indexes
 - Avoid unnecessary API calls
 - Debounce search
 - Optimize image display
 - Use pagination for exports where necessary
 - Avoid fetching all records into React
-

48. ACCESSIBILITY

Follow basic accessibility standards.

Use:

- Proper labels
- Keyboard navigation
- Accessible buttons
- Focus states
- Appropriate contrast
- Alt text for photos
- Accessible modal dialogs

49. RESPONSIVE MOBILE VIEW

On mobile:

- Sidebar should become a drawer/menu.
 - Tables should become horizontally scrollable or responsive cards.
 - Forms should use one-column layout.
 - Buttons should remain usable.
 - Filters should stack vertically.
-

50. TESTING

Implement tests for important functionality.

Backend:

- Login
- Authentication
- Authorization
- State API
- District filtering
- Person CRUD
- Validation
- Search
- State filtering
- District filtering

Frontend:

- Login
 - Form validation
 - State/district dependency
 - Person listing
 - Filters
 - Role-based UI
-

51. IMPORTANT BUSINESS RULE

The most important relationship is:

STATE → DISTRICT → PERSON

Example:

State:

Andhra Pradesh

District:

Nellore

Person:

John

The database and API must maintain these relationships correctly.

A district belonging to another state must never be assigned to a person.

This validation must exist on both:

Frontend

AND

Backend.

52. USER EXPERIENCE FLOW

Admin Flow

Login

↓

Dashboard

↓

Add Person

↓

Enter Name

↓

Enter Phone

↓

Enter Email

↓

Enter Address

↓

Select State

↓

Load Districts

↓

Select District

↓

Upload Photo

↓

Save

↓

Person appears in listing

↓

Admin can search/filter/edit/delete/export.

Normal User Flow

Login

↓

Dashboard

↓

Person Listing

↓

Search

↓

Select State

↓

Select District

↓

View person

↓

Export if permitted

Normal user cannot add/edit/delete.

53. DESIGN STYLE

Use a professional corporate dashboard style.

Avoid:

- Excessive animations
- Overly colorful UI
- Unnecessary gradients
- Complicated navigation
- Huge cards
- Excessive decorative elements

Prioritize:

- Clean layout
 - Readability
 - Consistency
 - Speed
 - Professional appearance
-

54. RESPONSIBILITY OF THE AI DEVELOPER

Do not simply create mock screens.

Build the actual working application.

The following must work end-to-end:

Frontend

↓

API

↓

Backend

↓

Database

↓

Response

↓

Frontend

Do not use fake/static data except for seed/demo data.

55. DEVELOPMENT PROCESS

Follow this implementation order:

PHASE 1

Set up project structure.

PHASE 2

Set up PostgreSQL and Django.

PHASE 3

Create database models and migrations.

PHASE 4

Create State/District APIs.

PHASE 5

Create authentication and JWT.

PHASE 6

Create role-based authorization.

PHASE 7

Create Person CRUD APIs.

PHASE 8

Create photo upload.

PHASE 9

Create Next.js frontend.

PHASE 10

Create login page.

PHASE 11

Create dashboard.

PHASE 12

Create Person form.

PHASE 13

Implement dynamic State → District.

PHASE 14

Create Person listing.

PHASE 15

Implement search/filter/pagination.

PHASE 16

Implement edit/delete.

PHASE 17

Implement CSV export.

PHASE 18

Implement PDF export.

PHASE 19

Implement User Management.

PHASE 20

Implement testing and error handling.

PHASE 21

Optimize UI and responsiveness.

PHASE 22

Prepare production deployment configuration.

56. BEFORE CODING

First analyze the entire requirement.

Then create:

1. Architecture
2. Database ER design
3. API design
4. Frontend page structure
5. Component structure
6. Authentication flow
7. Authorization flow

After that begin implementation.

Do not repeatedly ask me for confirmation for every small implementation decision.

Make sensible engineering decisions while following the requirements.

57. QUALITY REQUIREMENT

The final application should be:

- Production-ready
- Secure
- Responsive
- Maintainable
- Modular
- Scalable
- Properly validated
- Properly documented

Do not generate unnecessary code.

Do not duplicate code.

Do not leave TODO placeholders for core functionality.

Do not use mock APIs for final functionality.

Do not skip backend validation.

Do not skip authorization.

58. FINAL DELIVERABLE

At the end provide:

1. Complete frontend source code.
2. Complete backend source code.
3. Database models.
4. Database migrations.
5. Seed data.
6. API documentation.
7. Environment variable example.
8. README.
9. Setup instructions.
10. Production deployment instructions.
11. Test instructions.
12. Sample API requests/responses.
13. Explanation of authentication.
14. Explanation of authorization.
15. Explanation of State → District → Person relationships.

Also provide a final checklist showing each requirement and whether it has been implemented.

FINAL INSTRUCTION

Start by analyzing the requirements and creating the architecture/database/API plan.

Then implement the application phase by phase.

Do not stop at designing the UI.

Build the complete working full-stack application with real database connectivity, authentication, authorization, CRUD, dynamic State/District selection, search, filtering, pagination, photo upload, CSV/PDF export, and role-based access control.

The final result should be suitable for demonstrating to a real freelance client.